

Estructuras de Datos y Algoritmos

Presentación del curso y entorno

Dr. Jorge Armando Ramírez Díaz

Universidad Nacional Autónoma de México
ENES Unidad Juriquilla
Licenciatura en Matemáticas para el Desarrollo

Propósito de la primera sesión

Objetivo

Conocer cómo se trabajará el curso.

Hoy

Evaluación, herramientas, dinámica de trabajo y primera práctica.

Siguiente paso

Empezar formalmente con abstracción y TDA.

Qué se espera del curso

El curso combina teoría, implementación y análisis.

Comprender

Fundamentos teóricos, pseudocódigo y pensamiento abstracto.

Implementar

Construir soluciones en Python con pruebas y control de versiones.

Analizar

Justificar complejidad, corrección y decisiones de diseño.

Mapa del curso

Sem.	Tema central	Evidencia
1	Introducción y TDA I	Práctica 0
2	TDA II	Práctica 1
3	Complejidad algorítmica	Práctica 2
4	Corrección y Arreglos	Práctica 3
5	Recursión I	Práctica 4
6	Recursión II	Práctica 5
7	Listas	Práctica 6
8	Pilas y colas	Parcial 1

Sem.	Tema central	Evidencia
9	Árboles y árboles binarios	Práctica 7
10	Árboles binarios de búsqueda	Práctica 8
11	Árboles balanceados	Práctica 9
12	Tablas hash y heaps	Práctica 10
13	Búsqueda y ordenamiento I	Práctica 11
14	Búsqueda y ordenamiento II y Graficas I	Práctica 12
15	Graficas II	Parcial 2 y Entrega de proyecto Defensa de proyecto
16	Proyecto final	

Componente	Peso
Examen parcial 1	25 %
Examen parcial 2	25 %
Prácticas	25 %
Proyecto final	25 %
Bonificacion por retos	Hasta +10 puntos finales

La calificacion final sera $Final = \min(100, calificacionbase + bonificacion)$

Reposición en periodo ordinario

Si la calificación final no es aprobatoria, el estudiante podrá reponer uno de los exámenes parciales. Después se recalculará la calificación final. Sólo si continúa siendo no aprobatoria podrá presentar la reposición del segundo parcial.

Prácticas

Las prácticas se entregan en Classroom 50 y se califican con pruebas automáticas visibles. Cada práctica tendrá un límite de envíos permitidos.

Criterio	Peso
Prueba de sanidad	10 %
Pruebas básicas	50 %
Pruebas límite	40 %

Importante 1

Cualquier intento de alterar el autograding o vulnerar la seguridad del entorno de evaluación invalida la práctica y derivará en medidas académicas.

Importante 2

No se aceptan envíos después de tiempo salvo previa autorización.

Retos de algoritmia

En cada sesión se realizarán dos retos breves por equipos en la plataforma omegaUp.

Dinámica

- Todos los equipos pueden participar en ambos retos.
- Pasa a validación el primer equipo elegible que obtenga una solución aceptada.

Validación

- Un integrante del equipo candidato, elegido al azar, explica el algoritmo utilizado.
- El profesor y los demás estudiantes podrán hacer preguntas.
- Si no logra defenderlo, el reto queda sin ganador.

Incentivo

- Cada integrante del equipo ganador recibe 1 punto sobre su calificación final.
- Máximo 1 punto por estudiante en cada sesión.
- Máximo 10 puntos durante el curso.

Sin IA durante los retos · El ganador del primer reto puede participar en el segundo, pero ya no es elegible para otro punto · La calificación final no puede exceder 100.

No se realizaran retos de algoritmia en las semanas de examen ni en la semana de entrega de proyecto.

Organización

- Equipos de 2 personas.
- Individual o equipos de 3 solo con autorización previa.

Criterio	Peso
Prueba de sanidad (visible)	0 %
Pruebas básicas (oculta)	40 %
Pruebas límite (oculta)	30 %
Análisis de complejidad	15 %
Documentación y calidad de código	15 %

Validación del proyecto final

La defensa no suma puntos extra; valida la calificación base del proyecto.

Criterio	Puntos
Explica propósito, entradas, salidas y flujo general	2
Justifica decisiones de implementación y estructuras usadas	2
Traza el código con un ejemplo y reconoce casos límite	2
Analiza complejidad	2
Hace una modificación pequeña o depura un error en vivo	2

8–10 valida 100 %; 6–7 topa a 70 %; 4–5 topa a 50 %; 0–3 topa a 0 %.

Los proyectos se daran a conocer en la semana 12.

Uso de IA

Se recomienda ampliamente usar IA como apoyo, pero el estudiante debe entender, probar y defender lo que entrega.

El uso relevante de IA debe declararse explícitamente en las prácticas y el proyecto final.

El uso de IA en los exámenes está prohibido.

Habrà cero tolerancia con el uso indebido y no declarado de IA.

Uso aceptable

- Explicar conceptos o fragmentos de código.
- Proponer casos de prueba que el estudiante revise.
- Apoyar la depuración de código desarrollado por el estudiante.
- Revisar claridad, estilo y documentación.
- Comparar alternativas, ventajas, desventajas y complejidad.
- Practicar explicaciones o preguntas para la defensa.

No aceptable

- Usar IA durante exámenes o retos.
- Solicitar o copiar una solución completa y entregarla como trabajo propio.
- Incorporar código, pruebas o análisis que no se comprendan o verifiquen.
- Ocultar uso relevante de IA.
- No poder explicar, trazar, justificar o modificar la solución entregada.
- Usar IA para vulnerar el entorno de evaluación o el autograding.

Uso relevante: aquel que influye directamente en el algoritmo, el código, las pruebas, el análisis de complejidad o la documentación entregada. La declaración deberá indicar qué herramienta se utilizó y para qué.

- Steven S. Skiena, *The Algorithm Design Manual*, 3a ed., Springer, 2020.
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest y Clifford Stein, *Introduction to Algorithms*, 4a ed., MIT Press, 2022.
- George T. Heineman, Gary Pollice y Stanley Selkow, *Algorithms in a Nutshell: A Practical Guide*, 2a ed., O'Reilly Media, 2016.

Herramienta	Uso en el curso
VS Code	Entorno de desarrollo.
Conda	Manejo de entornos y dependencias.
Python	Lenguaje de programación.
Git	Historial de cambios y trazabilidad del trabajo.
Classroom 50	Entrega de prácticas.
Asistente IA	Apoyo para desarrollo.

Configuración del entorno de trabajo

Instalar Miniconda

- 1 Descargar desde <https://www.anaconda.com/download>.
- 2 Omitir registro.
- 3 Seleccionar distribución.
- 4 Seguir instrucciones de instalación para su sistema operativo.
- 5 Abrir Anaconda Prompt (Windows) o terminal (Linux/Mac).
- 6 Verificar instalación con `conda -version`.
- 7 Crear entorno básico de Python.

```
conda create -n algorithms python=3.11
conda activate algorithms
pip install numpy
```

- 8 Instalar paquetes necesarios preferentemente con un solo administrador: `pip` o `conda`.

Instalar VS Code

- 1 Descargar desde <https://code.visualstudio.com/download>.
- 2 Seleccionar distribución.
- 3 Seguir instrucciones de instalación para su sistema operativo.
- 4 Instalar extensión de Python en VS Code.
- 5 Seleccionar carpeta de trabajo.
- 6 Seleccionar el entorno de Conda usando el comando Python: Select Interpreter en VS Code.
- 7 Verificar que el terminal integrado de VS Code use el entorno correcto.

Instalar Git

- 1 Descargar desde <https://git-scm.com/downloads>.
- 2 Seleccionar distribución.
- 3 Seguir instrucciones de instalación para su sistema operativo.
- 4 Verificar instalación con `git -version` en el terminal.
- 5 Crear cuenta en GitHub.
- 6 En GitHub, ir a Settings, Developer settings y crear un token clásico con permisos de repositorio. Seleccionar duración según las necesidades.
- 7 Cuando Git pida credenciales, usar el usuario de GitHub y el token como contraseña.
- 8 Realizar cursos de Git en <https://learngitbranching.js.org>: Principal, secuencia introductoria 1-4; Acelerando 1-4; Remota, Push y Pull 1-8.

Instalar Asistente de IA

- ➊ Desde VS Code, ir a extensiones y buscar GitHub Copilot, Codex, Claude Code, o Gemini Code.
- ➋ En caso de no tener cuenta, solicitar acceso a GitHub for Education usando correo institucional <https://github.com/education>.
- ➌ Instalar la extensión oficial.
- ➍ Iniciar sesión con tu cuenta.
- ➎ Configurar sandbox, modelo y razonamiento.

Acceso a Classroom 50

- ➊ Proporcionar al profesor el nombre de usuario de GitHub para el envío de invitación.
- ➋ Aceptar por correo la invitación para unirse a la organización “2026-Algoritmos”.
- ➌ Ir a <https://classroom50.org/> e iniciar sesión con GitHub.
- ➍ Verificar que pueden acceder al curso.

- Hacer commits pequeños.
- Probar antes de entregar.
- Leer los mensajes de error antes de pedir ayuda.
- Preguntar temprano cuando algo no funcione.
- Cuidar claridad, no solo que el código pase pruebas.